

COMP6252 Deep Learning Technologies

Lecture 21: Reinforcement Learning

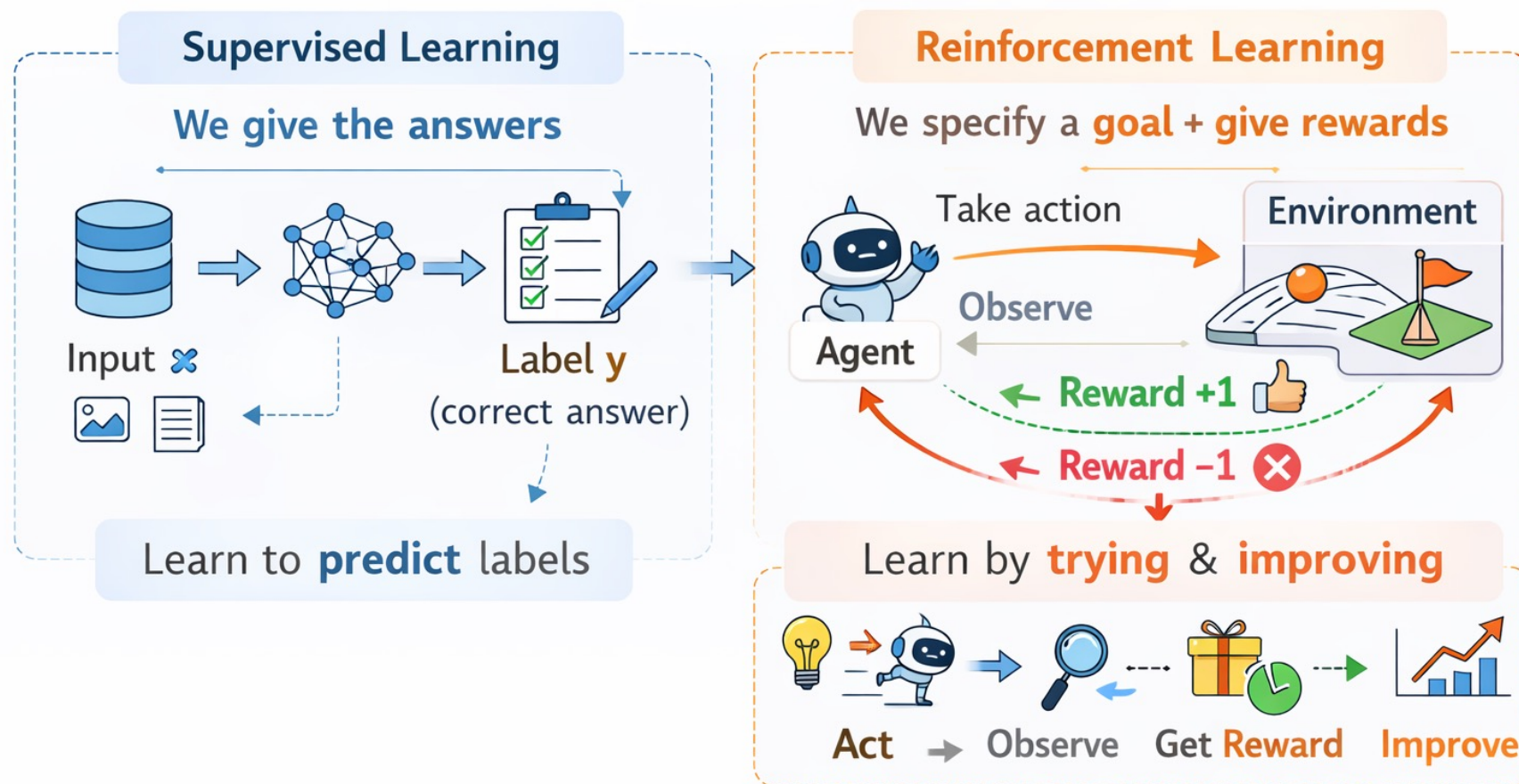
Zhiwu Huang

Zhiwu.Huang@soton.ac.uk

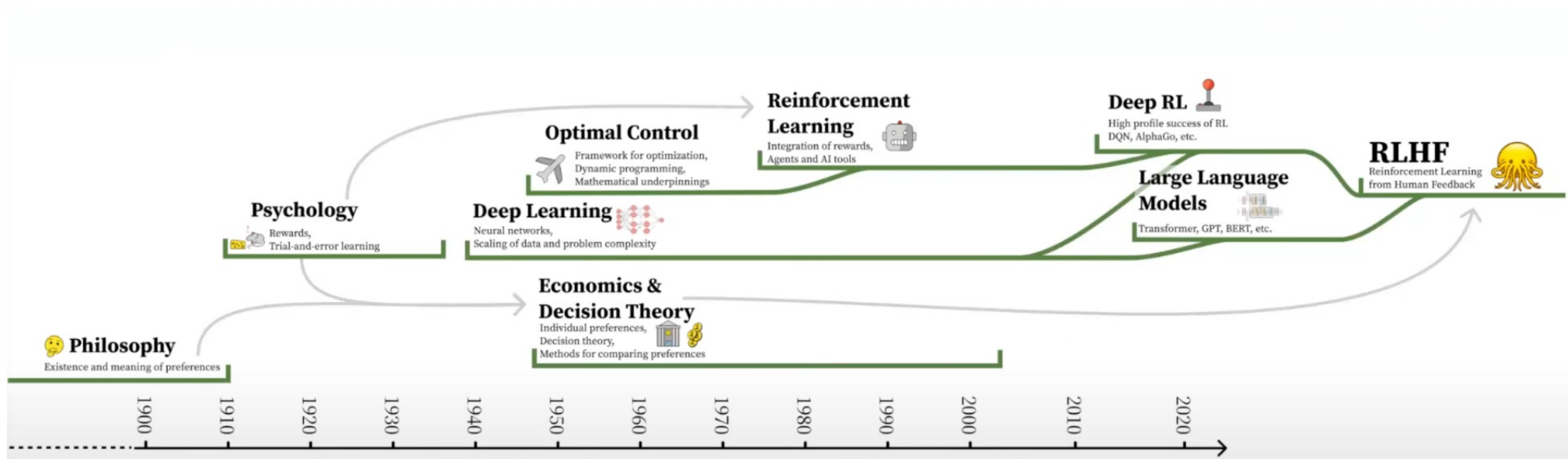
Lecturer (Assistant Professor) @ VLC of ECS
University of Southampton

[Credit: Xiaohao Cai (who designed the lecture backbone), David Silver, Twitter, Matt Gormley, Hado van Hasselt, Nathan Lambert, LeCun Yan etc.]

What is Reinforcement Learning?

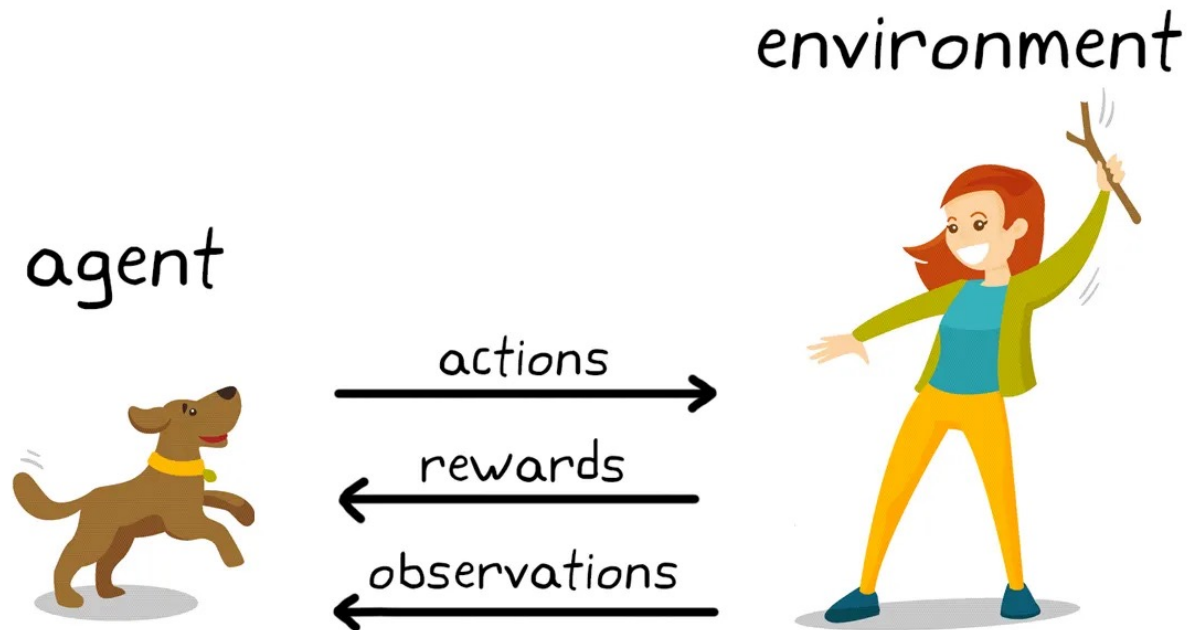


What is Reinforcement Learning?



<https://rlhfbook.com/c/10-preferences>

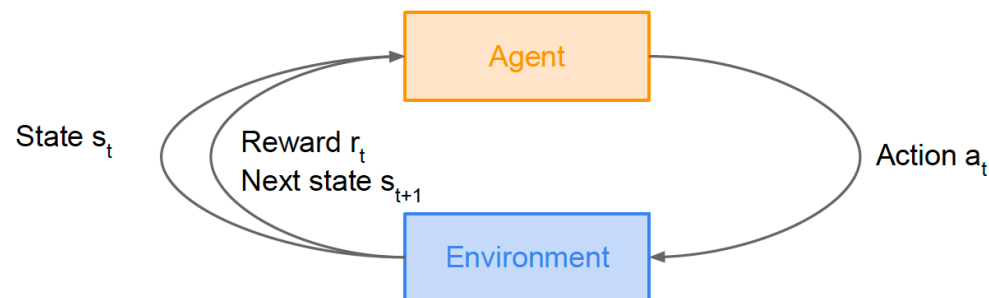
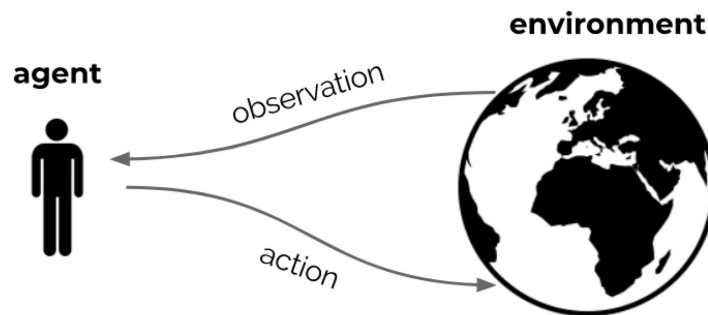
How Reinforcement Learning Works?



<https://medium.com/@shakthydoss/what-reinforcement-learning-really-means-for-agents-a84a0f2d7a99>

How Reinforcement Learning Works?

People and animals learn by interacting with our environment



Problems

involving an **agent** interacting with an **environment**, which provides numeric **reward** signals

Goal

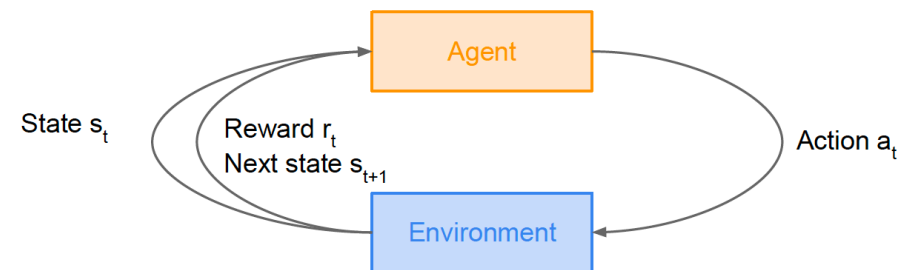
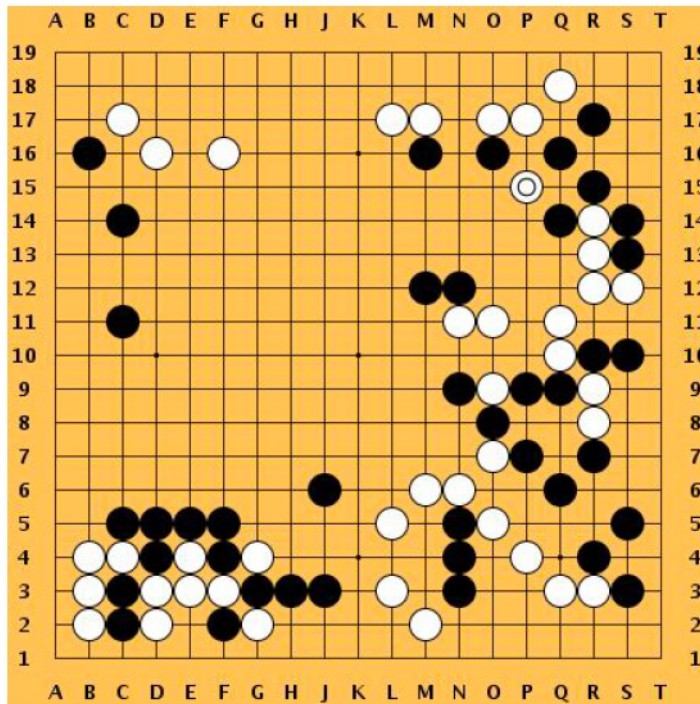
Learn a **policy**—how to take **actions**—in order to **maximise reward**

The reward hypothesis

Any goal can be formalised as the outcome of maximising a cumulative reward.

Examples of Reinforcement Learning

➤ GO



Objective: Win the game!

State: Position of all pieces

Action: Where to put the next piece down

Reward: 1 if win at the end of the game, 0 otherwise

Examples of Rewards

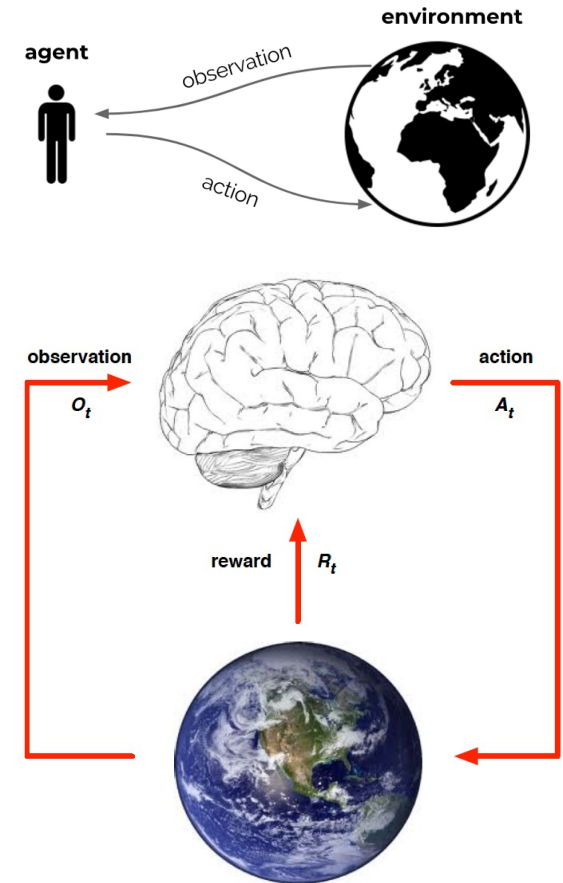
- Control a power station
 - +ve reward for producing power
 - -ve reward for exceeding safety thresholds
- Make a humanoid robot walk
 - +ve reward for forward motion
 - -ve reward for falling over

Formalising the RL Problem

➤ Agent, Environment, and Rewards

- ▶ At each step t the agent:
 - ▶ Receives observation O_t (and reward R_t)
 - ▶ Executes action A_t
- ▶ The environment:
 - ▶ Receives action A_t
 - ▶ Emits observation O_{t+1} (and reward R_{t+1})
- ▶ A **reward** R_t is a scalar feedback signal
- ▶ Indicates how well agent is doing at step t — defines the goal
- ▶ The agent's job is to maximize cumulative reward

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$$



Formalising the RL Problem

➤ History and State

- The **history** is the sequence of observations, actions, rewards

$$H_t = O_1, R_1, A_1, \dots, A_{t-1}, O_t, R_t$$

- i.e. all observable variables up to time t

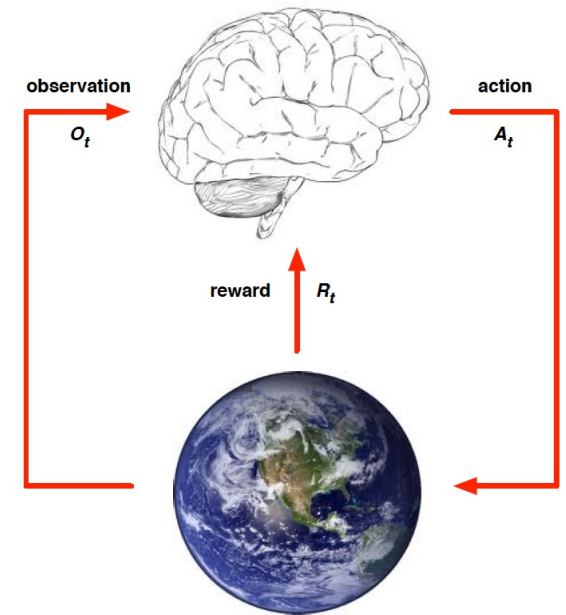
- What happens next depends on the history:

- The agent selects actions
- The environment selects observations/rewards

- **State** is the information used to determine what happens next

- Formally, state is a function of the history:

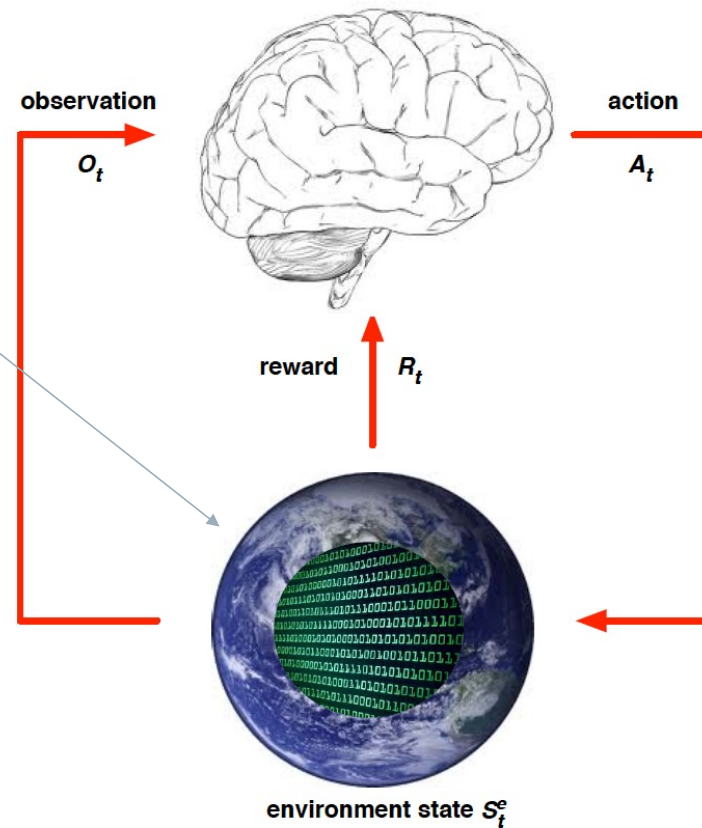
$$S_t = f(H_t)$$



Formalising the RL Problem

➤ Environment state

- The **environment state** S_t^e is the environment's private representation
- i.e. whatever data the environment uses to pick the next observation/reward
- The environment state is not usually visible to the agent

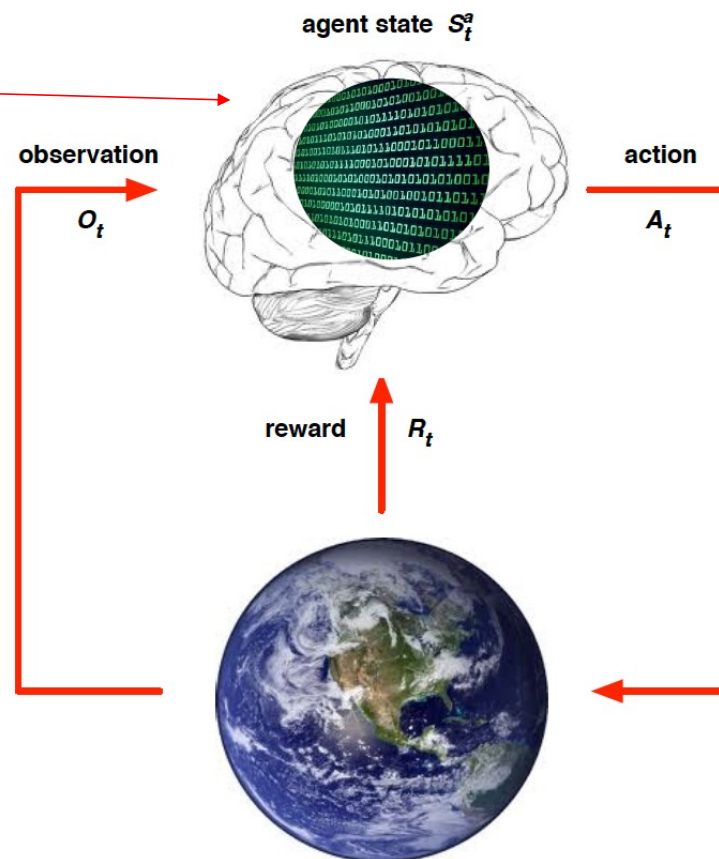


Formalising the RL Problem

➤ Agent state

- The **agent state** S_t^a is the agent's internal representation
- i.e. whatever information the agent uses to pick the next action
- i.e. it is the information used by reinforcement learning algorithms
- It can be any function of history:

$$S_t^a = f(H_t)$$



Formalising the RL Problem

➤ Fully observable environments

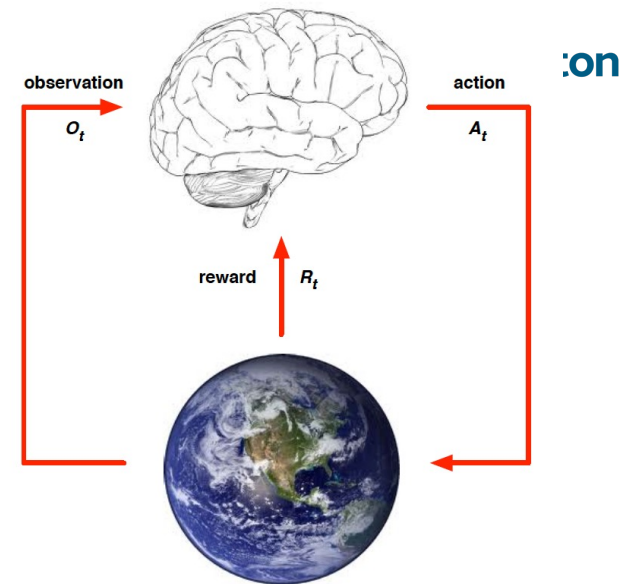
Full observability: agent **directly** observes environment state

$$O_t = S_t^a = S_t^e$$

- Agent state = environment state = information state
- Formally, this is a **Markov decision process** (MDP)

Partially
observable
environment

- **Partial observability:** agent **indirectly** observes environment:
 - A robot with camera vision isn't told its absolute location
 - A trading agent only observes current prices
 - A poker playing agent only observes public cards



A state S_t is **Markov** if and only if

$$\mathbb{P}[S_{t+1} \mid S_t] = \mathbb{P}[S_{t+1} \mid S_1, \dots, S_t]$$

Reinforcement Learning

➤ Policy function

- A **policy** is the agent's behaviour
- It is a map from state to action, e.g.
- Deterministic policy: $a = \pi(s)$
- Stochastic policy: $\pi(a|s) = \mathbb{P}[A_t = a | S_t = s]$

➤ Value function

- Value function is a prediction of future reward
- Used to evaluate the goodness/badness of states
- And therefore to select between actions, e.g.

$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | S_t = s]$$

Method	Used for	Pros	Cons
Monte Carlo (MC)	Episodic tasks	Unbiased, simple	High variance, slow
TD Learning (TD(0), SARSA, Q-learning)	Both episodic & continuous	Lower variance, fast updates	Biased, needs tuning
Neural Networks (DQN, DDPG, PPO)	Large state spaces	Handles high-dimensional states	Needs large data, complex

➤ Environment Model

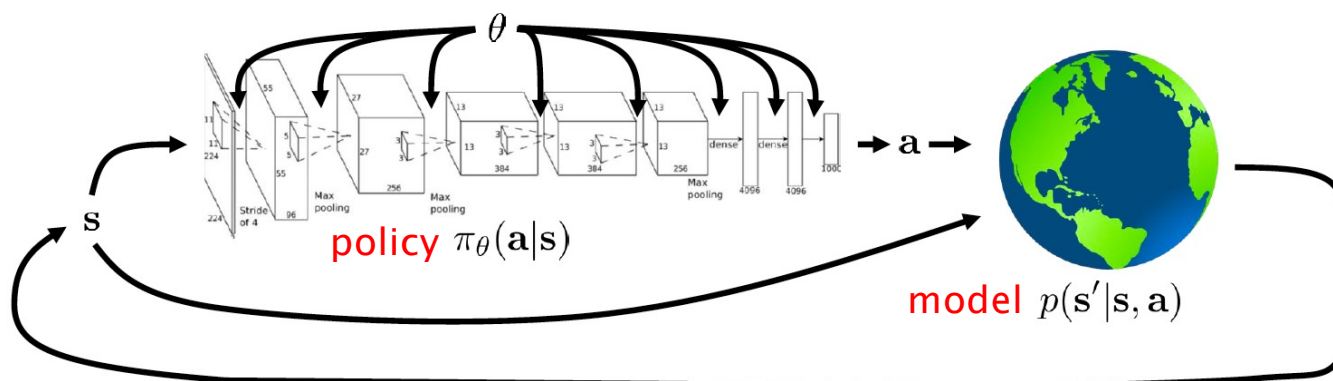
- A **model** predicts what the environment will do next
- $\mathcal{P}$ predicts the next state

$$\mathcal{P}_{ss'}^a = \mathbb{P}[S_{t+1} = s' | S_t = s, A_t = a]$$

$$\pi_*(s) = \operatorname{argmax}_{\pi} v_{\pi}(s)$$

Temporal Difference (TD) Learning

Deep Reinforcement Learning



The standard objective function in reinforcement learning is:

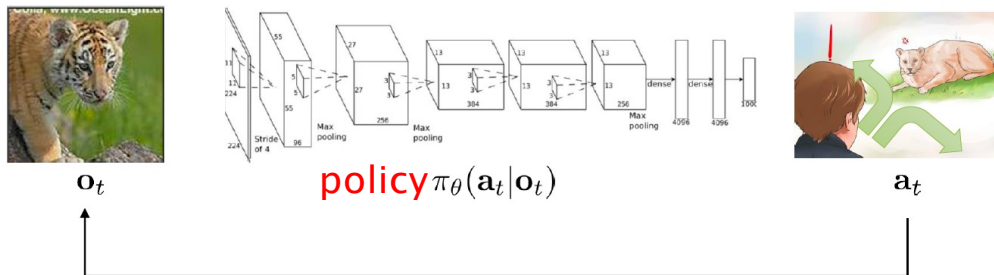
$$\underbrace{p_{\theta}(s_1, a_1, \dots, s_T, a_T)}_{p_{\theta}(\tau)} = p(s_1) \prod_{t=1}^T \underbrace{\pi_{\theta}(a_t | s_t) p(s_{t+1} | s_t, a_t)}_{\text{Markov chain on } (s, a)}$$

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^T \gamma^t r_t \right] \quad \text{Value}$$

where:

- $\tau = (s_1, a_1, s_2, a_2, \dots, s_T, a_T)$ is a **trajectory** (sequence of states and actions).
- $p_{\theta}(\tau)$ is the probability of a trajectory under policy π_{θ} .
- r_t is the **reward** at time t .
- $\gamma \in (0, 1]$ is the **discount factor**, which gives more weight to immediate rewards.
- $J(\theta)$ is the **expected cumulative reward**, which we aim to maximize.

Deep Reinforcement Learning



s_t – state
 o_t – observation
 a_t – action

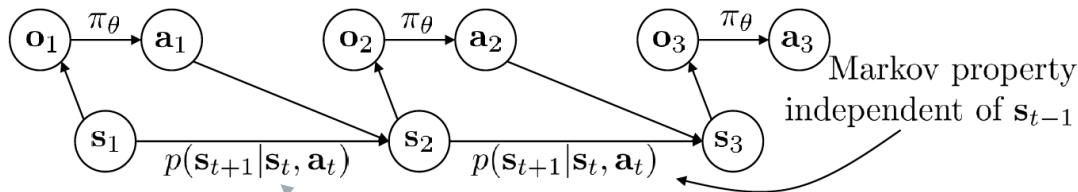
$\pi_\theta(a_t | o_t)$ – policy
 ~~$\pi_\theta(a_t | s_t)$ – policy (fully observed)~~

Observation Function $o_t = O(s_t)$

- The environment provides a true state s_t , but the agent does not directly access it.
- Instead, it receives an **observation** o_t , which is a function of s_t .
- This function can introduce **noise, occlusion, sensor limitations, or information loss**.

Examples of Observation Transformation

- **Robotics:**
 - s_t = The exact positions and velocities of all objects in a room.
 - o_t = A camera image with **occlusions, limited field of view, or sensor noise**.
- **Self-Driving Cars:**
 - s_t = Precise locations of all nearby vehicles, pedestrians, and road signs.
 - o_t = What the **LIDAR and cameras perceive**, which might be limited by fog, glare, or sensor resolution.
- **Game Playing (Atari, Chess, Go, etc.):**
 - s_t = The full board state, including the opponent's hidden strategy.
 - o_t = Only the **visible part of the board** or past moves without knowing the opponent's plan.



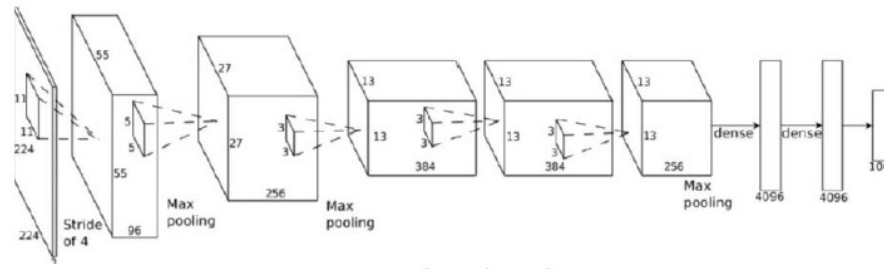
model $\mathcal{P}_{ss'}^a = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$

Imitation Learning

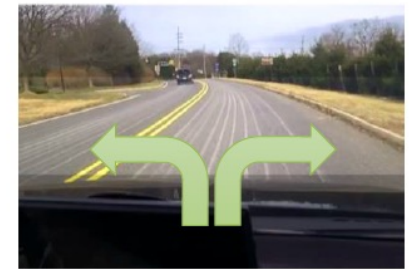
The process of teaching a model by demonstrating the correct behavior rather than specifying the rules explicitly.



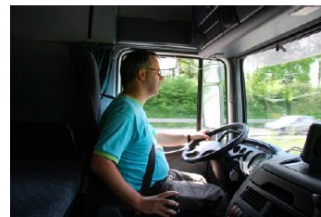
$\mathbf{o}_t$



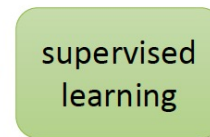
$\pi_{\theta}(\mathbf{a}_t|\mathbf{o}_t)$



$\mathbf{a}_t$



$\mathbf{o}_t$
 $\mathbf{a}_t$

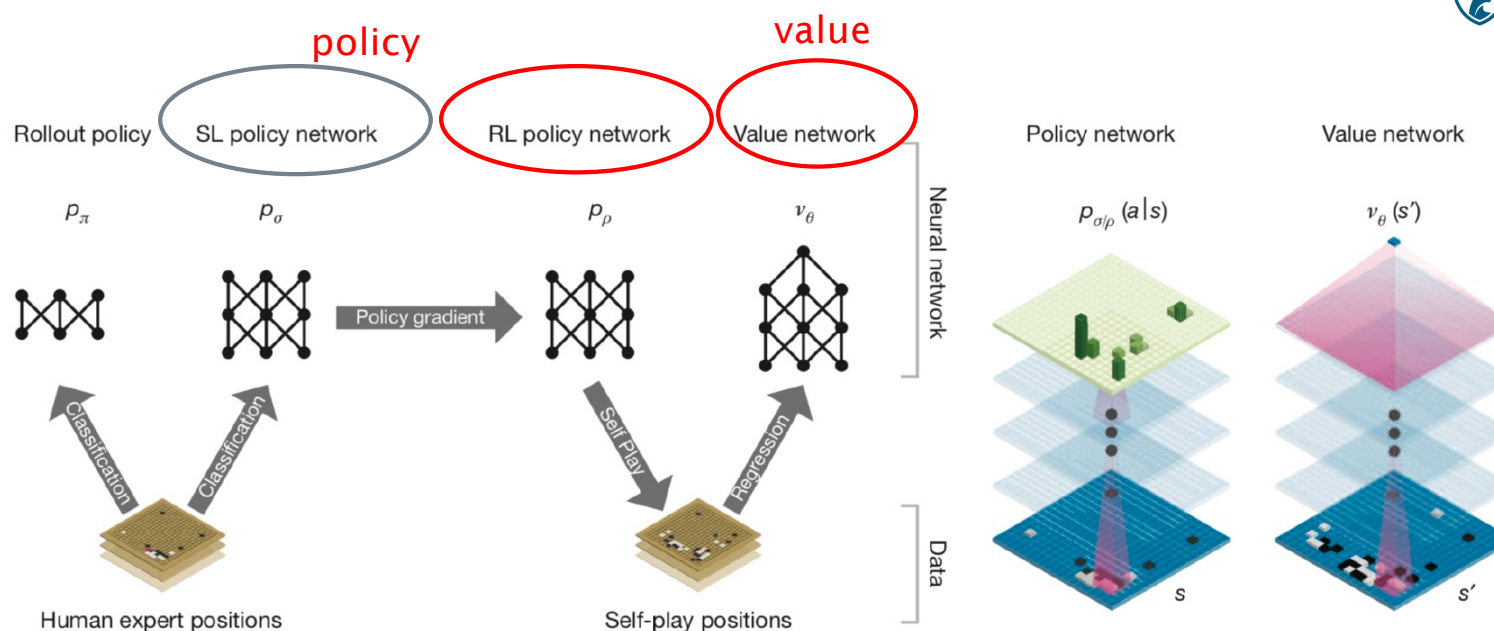


$\pi_{\theta}(\mathbf{a}_t|\mathbf{o}_t)$

Alpha Go

- State space is too large to represent explicitly since # of sequences of moves is $O(b^d)$
 - Go: $b=250$ and $d=150$
 - Chess: $b=35$ and $d=80$
- Key idea:
 - Define a neural network to approximate the value function
 - Train by policy gradient

Alpha Go

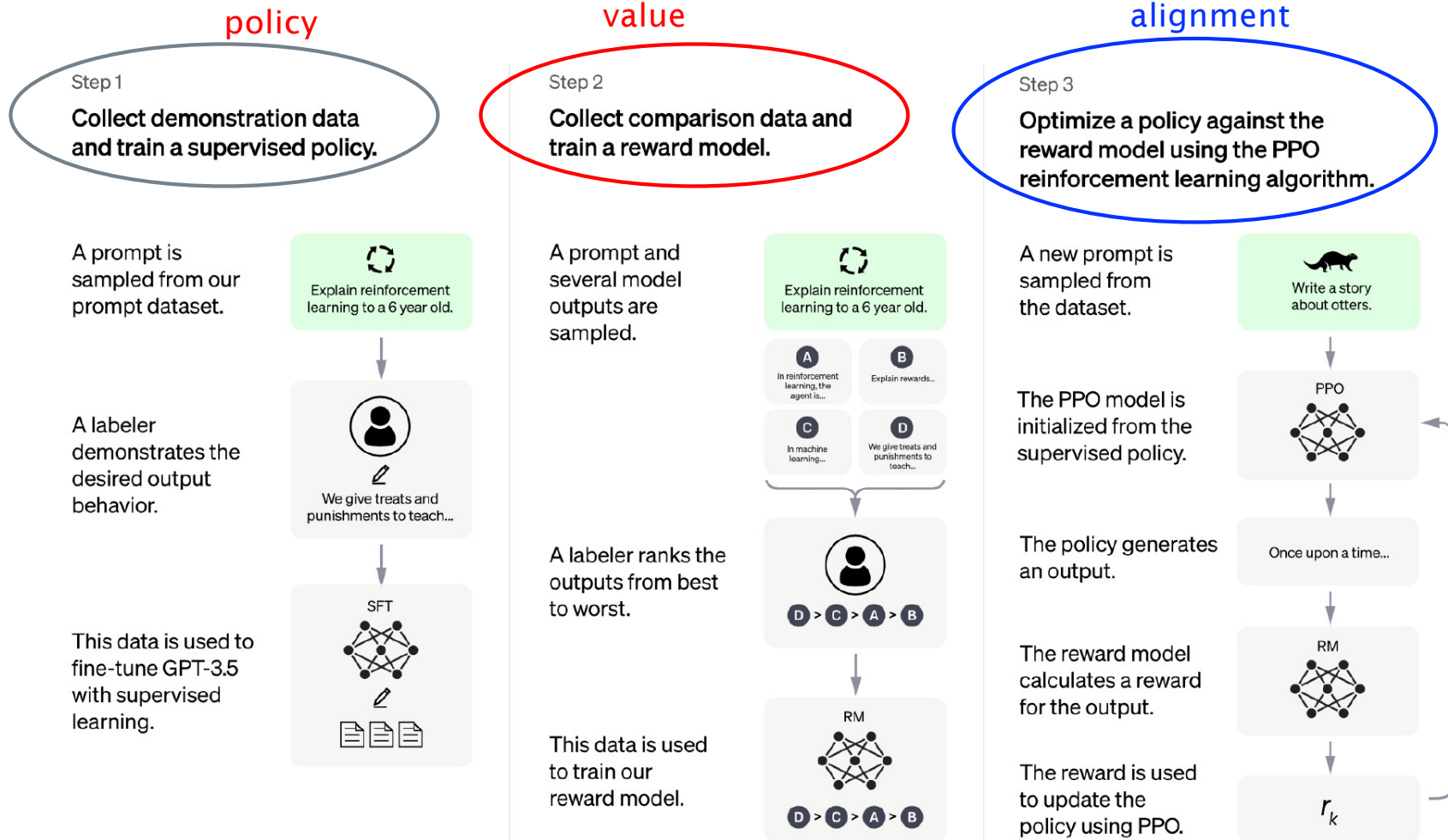


model $\mathcal{P}_{ss'}^a = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$

Network	Objective	Training Method	Loss Function	Key Idea
SL Policy Network	Mimic human expert moves	Supervised learning on expert games	Cross-entropy loss	Learn from human play
RL Policy Network	Improve policy by self-play	Reinforcement learning (policy gradient)	Policy gradient loss	Optimize for winning games
Value Network	Predict probability of winning from a state	Self-play with regression	Mean squared error (MSE)	Estimate board position strength

ChatGPT

Proximal Policy Optimization (PPO)



Learning Paradigms

[Slide from Matt Gormley]

Paradigm	Data
Supervised	$\mathcal{D} = \{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^N \quad \mathbf{x} \sim p^*(\cdot) \text{ and } y = c^*(\cdot)$
↔ Regression	$y^{(i)} \in \mathbb{R}$
↔ Classification	$y^{(i)} \in \{1, \dots, K\}$
↔ Binary classification	$y^{(i)} \in \{+1, -1\}$
↔ Structured Prediction	$\mathbf{y}^{(i)}$ is a vector
Unsupervised	$\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^N \quad \mathbf{x} \sim p^*(\cdot)$
Semi-supervised	$\mathcal{D} = \{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^{N_1} \cup \{\mathbf{x}^{(j)}\}_{j=1}^{N_2}$
Online	$\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots\}$
Active Learning	$\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^N$ and can query $y^{(i)} = c^*(\cdot)$ at a cost
Imitation Learning	$\mathcal{D} = \{(s^{(1)}, a^{(1)}), (s^{(2)}, a^{(2)}), \dots\}$
Reinforcement Learning	$\mathcal{D} = \{(s^{(1)}, a^{(1)}, r^{(1)}), (s^{(2)}, a^{(2)}, r^{(2)}), \dots\}$
Imitation Learning	$\mathcal{D} = \{(s^{(1)}, a^{(1)}, r^{(1)}), (s^{(2)}, a^{(2)}, r^{(2)}), \dots\}$



Answers given



Goal + reward